

Mapping ORM into OWL

Speaking the language of business experts

[Mustafa Jarrar](#)

Vrije Universiteit Brussel

2005

Overview

In many domains the ontology building process is difficult and time consuming. From practical experience, this is not because these domains are not well understood or a consensus cannot be made. But typically because it is difficult for domain experts to understand and use ontology languages. Indeed, current ontology languages and tools still require an understanding of its underpinning logics. Ontologies are being built by knowledgeable people who have the art of ontology engineering as well as the domain knowledge; or, by ontology engineers who elicit some knowledge by interviewing experts. Our motivation is to how enable business domain experts to build ontologies starting from the language they speak, and the “logic” they think of. This is not only to decrease the ontology development costs but also the quality of the ontological content will be better as its built and validated by the domain experts them self.

Limitations of ontology languages

The limitation of ontology languages that we present here is not in the sense of lack of more expressiveness or logical operator but the capability of these languages to be used by the subject matter experts. As analogy, think of the difference between the cost and the skills needed to program in Assembly verses C++ verses Visual Basic. Notice also that these languages are built on top of each other, i.e. C++ using Assembly and Visual Basic using C++.

Requirements and opportunities

For an ontology language to be easily understood by domain experts:

- Its constructors should be close to the language that business experts speak and understand. For example, it is easier for domain experts to say that “*it is mandatory for each book to have an ISBN*”, than to say “*the cardinality between the concept book and the concept ISBN is (1:0)*”; or to say that a person is identified by his passport number than to say the cardinality between them is (1:1). Notice that the notion of mandatory and identity can be specified by the notion of cardinality.
- it should have a graphical notation to enable quick and simple modeling. With a graphical notation here we do not mean merely a visualization, but a “graphical language” that allow domain experts to construct an ontology using a graphical notation for concept, relation, and axiom. In other words, such a language should

guide experts to “think” conceptually while building an ontology. notice that the user of current ontology modeling tools think outside these tools (maybe drawing on papers, etc.), and enter their results to the tools. Ontology visualization in such tools is very simplistic (e.g. only connection between concepts), and only used to view, rather than to model, knowledge.

Instead of inviting new ontology languages to meet such requirements, i.e. that business people can understand, we propose to investigate how existing business languages and graphical notations can be reused for ontology engineering purposes. Such languages could be the Object Role Modeling (ORM), UML diagrams (not only the class-diagram), etc¹.

Our experiment on defining a full mapping of ORM into description logics is very promising². See an example of an axiomatization built using ORM in figure X1. We have found that almost³ all ORM constructs can be mapped into description logic, and so to OWL-DL. We believe that such mapping will enable the inheritance and reuse of the ORM literature, tools, techniques, applications and users, etc.

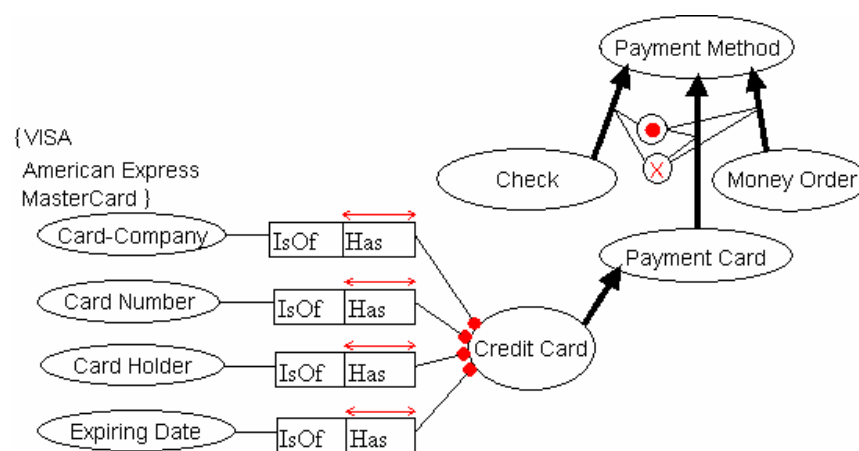


Fig. x1. An example of ORM model.

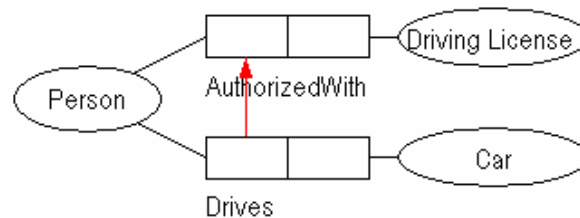
ORM is being used by business experts, specially in the database and business rules communities. ORM is a successor of the NIAM (Natural-language Information Analysis Method, developed early 70s). It was explicitly designed to play the role of a stepwise methodology: to arrive at the “semantics” of a business application’s data based on natural language communication between domain experts. See for example the verbalization of the “subset” constraint in figure X2, which is generated automatically. ORM is not only a conceptual modeling language but also comprehensive in its treatment

¹ Many other business notations could be found in Microsoft Visio.

² This experiment was done during a Kweb research exchange between the Free University of Brussels (VUB) and the Free University of Bolzano (FUB).

³ Except few ORM constraint types (but they rarely used in the ORM practice) such as external frequency, external uniqueness, and some of the ring constraints.

of many "practical" or "standard" business rules and constraint types (e.g. identity, mandatory, uniqueness, subsumption, subset, equality, exclusion, value, frequency, symmetric, intransitive, acyclic, etc.). Furthermore, ORM has an expressive and stable graphical notation since it captures many rules graphically and it minimizes the impact of change on the models.



“If a *Person* *Drives* a *Car* then this *Person* must be *AuthorizedWith* a *Driving License*”

Fig. x2. An example of a subset constraint verbalized (automatically) into a natural language sentence.

To sum up, the mapping of business languages and graphical notations into description logic based languages has the following immediate advantages:

- 1- allow business domain experts to participate in the ontology modeling and validation processes;
1. enable the reuse of the business literature (such as tools, methods, applications, users, best practice, etc.) in other words, current ontology languages will be moved from the mathematical sense into “business logics”;
- 2- is a good way to test and improve description logic based languages them self, as this mapping will discover which logical operator that are needed and used in industry.

We shall continue to instigate and incorporate this kind of language extensions in the third and the forth JPAs of WP2.5.